



دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

مبانی و کاربردهای هوش مصنوعی

پروژه سوم (Neon Hunt)

استاد: دکتر مرضیه حسینی

دستیاران آموزشی:

فاطمه صیادزاده

مرضیه کرمی

مسیح روغنی

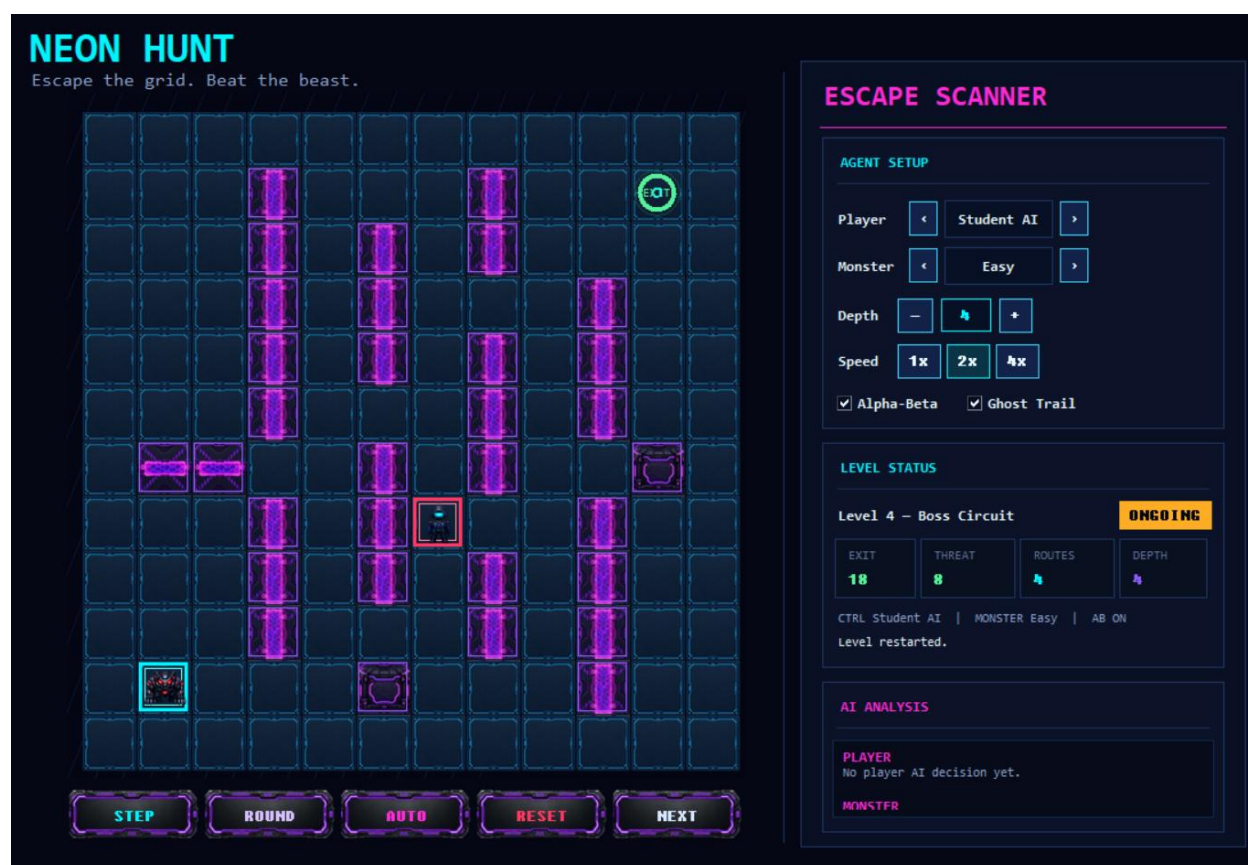
نیم سال دوم ۱۴۰۴-۱۴۰۵

۱. مقدمه و هدف بازی

در بازی Neon Hunt شما کنترل شخصیت Hacker را بر عهده دارید. هدف Hacker این است که در یک شبکه‌ی نئونی از دست Cyber Beast فرار کند و به خانه‌ی خروج برسد.

در هر مرحله، Hacker نقش Player را دارد و Cyber Beast نقش Monster را بازی می‌کند. باید با انتخاب حرکات‌های مناسب، به Exit برسد یا حداقل تا پایان مرحله زنده بماند. بعد از حرکت Player حرکت می‌کند و تلاش می‌کند Player را بگیرد.

هدف این تمرین این است که برای Player یک هوش مصنوعی طراحی و پیاده‌سازی کنید که بتواند با استفاده از Evaluation Function، Minimax و Alpha-Beta Pruning تصمیم‌گیری کند.



شکل ۱ — محیط بازی Neon Hunt شامل زمین بازی، پنل Escape Scanner و دکمه‌های کنترل

۱.۱ اجزای اصلی بازی

محیط بازی از چند بخش اصلی تشکیل شده است:

زمین بازی

زمین بازی به صورت یک grid نمایش داده می‌شود. هر خانه در این grid می‌تواند یکی از حالت‌های زیر را داشته باشد:

- مسیر قابل حرکت
- مانع یا دیوار
- خانه‌ی خروج
- محل قرارگیری Player
- محل قرارگیری Monster

Player و Monster فقط می‌توانند در چهار جهت اصلی حرکت کنند:

UP / DOWN / LEFT / RIGHT

حرکت به بیرون نقشه یا ورود به خانه‌های دیوار مجاز نیست.

پنل *Escape Scanner*

پنل سمت راست بازی با عنوان *Escape Scanner* اطلاعات مهم وضعیت بازی را نمایش می‌دهد. این پنل شامل موارد زیر است:

- حالت کنترل Player
- سطح سختی Monster
- عمق جست‌وجوی Player
- سرعت اجرای بازی
- وضعیت مرحله
- فاصله Player تا Exit
- فاصله Player تا Monster
- تعداد مسیرهای فرار
- خلاصه‌ای از تصمیم‌های AI

گزینه Player

در بخش Player دو حالت وجود دارد:

Human

در این حالت، شما به صورت دستی Player را کنترل می کنید.

Student AI

در این حالت، فایل AI دانشجو تصمیم می گیرد. برای تست کد خودتان باید Player را روی Student AI قرار دهید.

گزینه Monster

Monster چند سطح سختی دارد:

Easy

Normal

Hard

Boss

هرچه سطح سختی بالاتر باشد، Monster تصمیم گیری قوی تری خواهد داشت.

گزینه Depth

Depth عمق جست و جوی الگوریتم شما را مشخص می کند. هرچه Depth بیشتر باشد، AI آینده ی بیشتری را بررسی می کند؛ اما اجرای الگوریتم نیز کندتر خواهد شد.

گزینه Alpha-Beta

اگر گزینه Alpha-Beta فعال باشد، تابع `alpha_beta` شما برای تصمیم گیری استفاده می شود. اگر این گزینه غیرفعال باشد، تابع `minimax` استفاده می شود.

گزینه Ghost Trail

این گزینه مسیرهای اخیر حرکت را روی `grid` نمایش می دهد و برای بررسی رفتار AI مفید است.

گزینه Speed

Speed سرعت اجرای خودکار بازی را مشخص می‌کند:

1x

2x

4x

دکمه‌های پایین صفحه

STEP فقط یک حرکت Player انجام می‌شود.

ROUND یک دور کامل انجام می‌شود؛ یعنی ابتدا Player و سپس Monster حرکت می‌کند.

AUTO بازی به صورت خودکار ادامه پیدا می‌کند.

RESET مرحله از ابتدا شروع می‌شود.

NEXT مرحله‌ی بعدی باز می‌شود.

۲. فایل تحویلی دانشجو

فایل تحویلی شما باید یک فایل Python باشد. نام پیشنهادی فایل:

student_player_ai.py

در ابتدای فایل باید کد دانشجویی خود را وارد کنید:

```
STUDENT_ID = "1234567890"
```

کد دانشجویی باید دقیقاً ۱۰ رقم و فقط عدد باشد.

۳. توابعی که باید پیاده‌سازی شوند

در این پروژه شما فقط باید چهار تابع زیر را کامل کنید:

```
evaluate(state)
```

```
minimax(state, depth, maximizing_player, stats=None)
```

```
alpha_beta(state, depth, alpha, beta, maximizing_player, stats=None)
```

```
choose_player_move(state, depth, use_alpha_beta=True)
```

توابع مربوط به قوانین پایه‌ی بازی آماده هستند و لازم نیست آن‌ها را پیاده‌سازی کنید.

۴. تابع `evaluate(state)`

تابع `evaluate` وظیفه دارد وضعیت فعلی بازی را از دید `Player` یا `Hacker` امتیازدهی کند. در این تابع، به هر `state` یک مقدار عددی نسبت داده می‌شود؛ به طوری که امتیاز بزرگ‌تر نشان‌دهنده وضعیت بهتر برای `Hacker` باشد.

این امتیاز باید نشان دهد که وضعیت فعلی تا چه اندازه برای رسیدن به خروج و زنده ماندن مناسب است. برای طراحی این تابع می‌توانید عواملی مانند فاصله تا خروج، میزان خطر `Monster`، امکان فرار از موقعیت فعلی و وضعیت‌های برد یا باخت را در نظر بگیرید.

انتخاب این عوامل و نحوه وزن‌دهی به آن‌ها بخش مهمی از طراحی `AI` شماست و می‌تواند تأثیر زیادی بر کیفیت تصمیم‌گیری `Player` داشته باشد.

۴.۱. توابع کمکی مفید برای `evaluate`

در فایل `template` این توابع برای شما `import` شده‌اند:

```
bfs_distance(state, start, goal)
```

```
escape_routes(state, position)
```

مثال استفاده:

```
d_exit = bfs_distance(state, state.player, state.exit)
```

```
d_monster = bfs_distance(state, state.player, state.monster)
```

```
routes = escape_routes(state, state.player)
```

۵. تابع `minimax(state, depth, maximizing_player, stats=None)`

این تابع باید الگوریتم Minimax عمق محدود را از دید Player پیاده‌سازی کند.

قواعد اصلی:

- وقتی `maximizing_player=True` است، نوبت Player است و باید امتیاز را بیشینه کند.
- وقتی `maximizing_player=False` است، نوبت Monster است و باید امتیاز Player را کمینه کند.
- اگر `depth == 0` باشد، باید `evaluate(state)` برگردانده شود.
- اگر بازی تمام شده باشد، باید `evaluate(state)` برگردانده شود.

اگر `stats` داده شده بود، بهتر است تعداد `state` های بررسی شده را ثبت کنید:

if stats is not None:

```
stats["states_explored"] = stats.get("states_explored", 0) + 1
```

۶. تابع `alpha_beta(state, depth, alpha, beta, maximizing_player, stats=None)`

این تابع نسخه‌ی بهینه‌تر Minimax با هرس Alpha-Beta است. پیاده‌سازی این تابع اجباری است و بخشی از نمره‌ی بخش Minimax / Alpha-Beta را تشکیل می‌دهد. در Grader، خروجی این تابع باید از نظر مقدار تصمیم‌گیری با Minimax سازگار باشد. هدف Alpha-Beta این است که همان مقدار Minimax را برگرداند، اما بخشی از شاخه‌های غیرضروری درخت جست‌وجو را بررسی نکند.

اگر شاخه‌ای `prune` شد، می‌توانید این مقدار را افزایش دهید:

if stats is not None:

```
stats["pruned_branches"] = stats.get("pruned_branches", 0) + 1
```

نکته مهم:

تابع `alpha_beta` باید از نظر مقدار نهایی با تابع `minimax` سازگار باشد.

۷. تابع `choose_player_move(state, depth, use_alpha_beta=True)`

این تابع باید بهترین حرکت Player را انتخاب کند.

برای هر حرکت قانونی Player باید مراحل زیر انجام شود:

۱. حرکت را اعمال کنید.

۲. وضعیت حاصل را با Minimax یا Alpha-Beta ارزیابی کنید.

۳. بهترین حرکت را انتخاب کنید.

خروجی پیشنهادی تابع:

```
{  
    "best_move": "UP",  
    "scores": {  
        "UP": 12.5,  
        "DOWN": -3.2  
    },  
    "states_explored": 120,  
    "pruned_branches": 15,  
    "principal_variation": []  
}
```

حداقل خروجی لازم این است که `best_move` معتبر داشته باشید.

۸. توابع آماده

توابع زیر در فایل `student_support.py` آماده هستند:

`neon_hunt/ai/student_support.py`

و در template شما import شده‌اند:

```
from neon_hunt.ai.student_support import get_possible_moves, apply_move, is_terminal
```

۸.۱. تابع `get_possible_moves(state, agent)`

این تابع حرکت‌های قانونی `agent` را برمی‌گرداند.

مثال:

```
moves = get_possible_moves(state, AGENT_PLAYER)
```

۸.۲. تابع `apply_move(state, move, agent)`

این تابع حرکت را اعمال می‌کند و یک `state` جدید برمی‌گرداند.

این تابع `state` اصلی را تغییر نمی‌دهد.

مثال:

```
next_state = apply_move(state, "UP", AGENT_PLAYER)
```

۸.۳. تابع `is_terminal(state)`

این تابع وضعیت پایان بازی را مشخص می‌کند.

خروجی این تابع یکی از موارد زیر است:

ONGOING یا MONSTER_WIN یا PLAYER_WIN

۹. نکات مهم پیاده‌سازی

در پیاده‌سازی خود به نکات زیر توجه کنید:

- مسیر حل مرحله‌ها را `hardcode` نکنید.
- مختصات خاص نقشه‌ها را در کد ذخیره نکنید.
- تابع `evaluate` باید عمومی باشد و روی نقشه‌های مختلف کار کند.
- Greedy ساده، یعنی فقط نزدیک شدن به خروج، برای نمره خوب کافی نیست.
- Minimax و Alpha-Beta باید از دید Player نوشته شوند.
- Alpha-Beta باید همان جواب Minimax را تولید کند.

- اگر AI در یک مسیر رفت و برگشتی گیر کند، نمره Gameplay کاهش پیدا می‌کند.
- برای نمره خوب باید حداقل Minimax ، Alpha-Beta و Evaluation متوسط داشته باشید.
- برای نمره امتیازی باید رفتار AI در مرحله Boss هم قوی باشد.

۱۰. معیار نمره‌دهی

نمره اصلی از ۱۰۰ محاسبه می‌شود و مرحله Boss تا ۱۰ امتیاز اضافه دارد.

۱۰.۱. جدول کلی نمره‌دهی

بخش	امتیاز
Interface و کد دانشجویی	10
Evaluation Sanity	20
Minimax / Alpha-Beta	20
Gameplay Level 1 تا 3	50
Boss Bonus	+10

۱۰.۲. Interface — 10 score

در این بخش بررسی می‌شود که:

- STUDENT_ID معتبر باشد.
- evaluate عدد برگرداند.
- minimax عدد برگرداند.
- alpha_beta عدد برگرداند.
- choose_player_move حرکت قانونی برگرداند.

۱۰.۳. Evaluation Sanity — 20 score

در این بخش بررسی می‌شود که تابع ارزیابی شما موارد پایه را تشخیص دهد:

- برد Player خیلی بهتر از حالت عادی باشد.
- باخت Player خیلی بدتر از حالت عادی باشد.

- نزدیک‌تر بودن به خروج بهتر باشد.
- نزدیک بودن به Monster بدتر باشد.
- موقعیت امن‌تر بهتر از موقعیت خطرناک باشد.

۱۰.۴ . Minimax / Alpha-Beta — 20 score

در این بخش، هر دو تابع Minimax و Alpha-Beta بررسی می‌شوند. پیاده‌سازی Alpha-Beta اجباری است و باید از نظر مقدار نهایی و حرکت انتخاب‌شده با Minimax سازگار باشد. تفاوت اصلی Alpha-Beta این است که باید همان نتیجه‌ی Minimax را با بررسی state‌های کمتر به دست آورد.

- minimax باید با مقدار reference هم‌خوان باشد.
- alpha_beta باید با مقدار reference و خروجی minimax هم‌خوان باشد.
- alpha_beta باید در صورت امکان شاخه‌های غیرضروری را prune کند.
- choose_player_move باید با بهترین حرکت depth-limited minimax یا alpha_beta سازگار باشد.

۱۰.۵ . Gameplay — 50 score

امتیاز	سطح Monster	Level
12	Easy	Level 1
18	Normal	Level 2
20	Hard	Level 3

در بخش Gameplay، علاوه بر نتیجه‌ی نهایی مرحله، کیفیت رفتار AI نیز در Grader ارزیابی می‌شود. عواملی مانند رسیدن به خروج، افزایش امنیت Player، کاهش فاصله تا خروج، دور ماندن از Monster و پرهیز از حرکت‌های تکراری در امتیاز اثرگذار هستند. بنابراین، برد کامل مرحله بیشترین امتیاز را دارد، اما رفتارهای ناقص یا دارای loop بسته به میزان پیشرفت AI تنها بخشی از امتیاز را دریافت می‌کنند.

۱۰.۶. Boss Bonus — 10 score

امتیاز	سطح Monster	Level
+10	Boss	Level 4

این بخش امتیازی است. هرچه عملکرد AI شما در Grader بهتر باشد، امتیاز بیشتری دریافت می‌کنید.

گرفتن امتیاز کامل این بخش معمولاً نیازمند Evaluation قوی‌تر، جلوگیری از loop و تصمیم‌گیری خلاقانه‌تر است.

۱۱. سقف‌های نمره‌ای

برای جلوگیری از گرفتن نمره بالا با روش‌های خیلی ساده، Grader چند سقف دارد:

سقف نمره	وضعیت
حداکثر 45	اگر Level 1 حل نشود
حداکثر 55	اگر Evaluation خیلی ضعیف باشد
حداکثر 55	اگر Minimax یا Alpha-Beta درست نباشد
حداکثر 55	اگر Level 2 با شکست تمام شود
حداکثر 70	اگر choose_player_move با Minimax سازگار نباشد

بنابراین راه حل Greedy ساده برای نمره خوب کافی نیست.

۱۲. گزارش پایانی

علاوه بر فایل کد، باید یک گزارش کوتاه و منظم بنویسید.

کیفیت گزارش مهم است و در ارزیابی نهایی پروژه در نظر گرفته می‌شود.

در گزارش خود حتماً این ۶ بخش را توضیح دهید:

۱. نام و کد دانشجویی.

۲. ایده و منطق تابع evaluate.

۳. توضیح پیاده‌سازی Alpha-Beta و Minimax.

- ۴. توضیح روش جلوگیری از نزدیک شدن به Monster یا گیر کردن در loop .
- ۵. نتیجه تست‌های خودتان روی مرحله‌ها.
- ۶. توضیح ایده‌های اضافه برای بخش امتیازی Boss ، اگر پیاده سازی کرده‌اید.

۱۳. اجرای بازی

از ریشه‌ی پروژه دستور زیر را اجرا کنید:

```
python main.py
```

سپس در UI حالت Player را روی Student AI قرار دهید.

۱۴. نکات تحویل و ارائه

- پروژه می‌تواند به صورت انفرادی یا در گروه‌های حداکثر دو نفره انجام شود. در پروژه‌های گروهی، مشخصات هر دو دانشجو باید در گزارش نهایی ذکر شود.
- فایل تحویلی باید شامل کد تکمیل‌شده‌ی student_player_ai.py و یک گزارش نهایی در قالب PDF باشد. از ارسال فایل‌های اضافی، پوشه‌ی محیط بازی یا فایل‌های فشرده‌ی نامرتب خودداری کنید.
- گزارش نهایی باید شامل توضیح تابع Evaluation ، نحوه‌ی پیاده‌سازی Minimax و Alpha-Beta ، تحلیل رفتار AI در مراحل مختلف، بررسی علت شکست یا ایجاد loop و مقایسه‌ی عملکرد الگوریتم‌ها در عمق‌های مختلف باشد.
- نتایج اجرای AI در Level های مختلف و تنظیمات مورد استفاده، مانند Depth و سطح سختی Monster ، باید به صورت دقیق در گزارش ذکر شوند.
- در زمان ارائه‌ی شفاهی، دانشجو باید به تمام بخش‌های کد، منطق تابع Evaluation ، الگوریتم Minimax، هرس Alpha-Beta و نحوه‌ی انتخاب حرکت نهایی مسلط باشد.
- در صورت انجام پروژه به صورت گروهی، هر دو عضو گروه باید به کل پروژه و تمام توابع پیاده‌سازی‌شده مسلط باشند. تقسیم کد بین اعضا، مسئولیت پاسخ‌گویی هر دو نفر را کاهش نمی‌دهد.
- ممکن است در جلسه‌ی ارائه، تغییراتی در نقشه، عمق جست‌وجو یا سطح سختی Monster اعمال شود و عملکرد AI دوباره بررسی شود؛ بنابراین از hardcode کردن مسیر مراحل خودداری کنید.
- فایل‌های محیط بازی و Template اولیه از طریق GitHub در اختیار دانشجویان قرار می‌گیرد. در صورت وجود مشکل در دسترسی به GitHub ، فایل‌ها از طریق گروه کلاس نیز منتشر خواهند شد.
- مسئولیت اطمینان از اجرای صحیح پروژه پیش از تحویل بر عهده‌ی دانشجو است. فایل ارسالی باید بدون تغییر در ساختار پروژه و با اجرای دستور python main.py قابل آزمایش باشد.